

CHAPTER 3

栈和队列

Stacks, and Queues

Faculty of Information and Control Engineering

Huanliang Sun

Email: sunhl@sjzu.edu.cn

Office: Wu(戊2-306) 24690666

§ 3.1 栈ADT

1. ADT

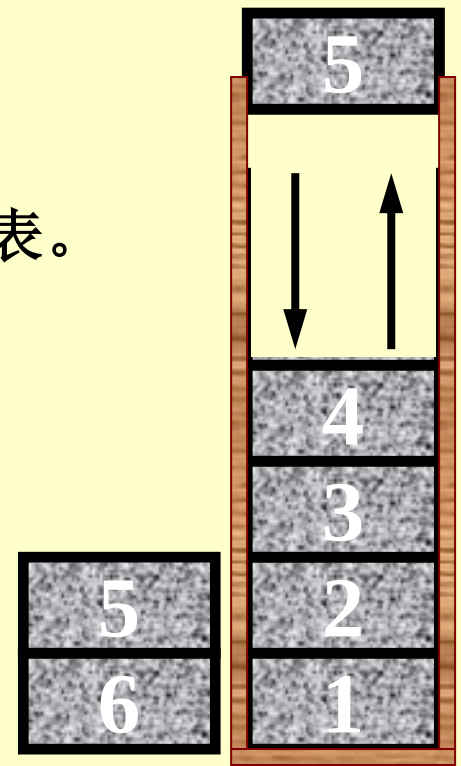
- 定义：限定仅在**表尾**进行插入或删除操作的线性表。

表尾——**栈顶**，是插入、删除端 (Top)。

表头——**栈底**。

不含元素的空表称**空栈**。

- 特点：先进后出 (**FILO**) 或后进先出 (**LIFO**)



Operations:

☞ Int **IsEmpty**(Stack S);

☞ Stack **CreateStack**();

☞ **ClearStack**(Stack S);

☞ **TraverseStack** (Stack S);

☞ **Push**(ElementType X, Stack S);进、入、压栈

☞ ElementType **Top**(Stack S);

☞ **Pop**(Stack S);出、弹栈

Note: A **Pop** (or **Top**) on an **empty** stack is an error in the stack ADT.

Push on a **full** stack is an implementation error but not an ADT error.

2. 栈实现

➤ 链栈 Linked List Implementation (with a header node)

👉 **Push:** ① `TmpCell = (Stack)malloc(sizeof(SNode));`

② `TmpCell->Next = S->Next`

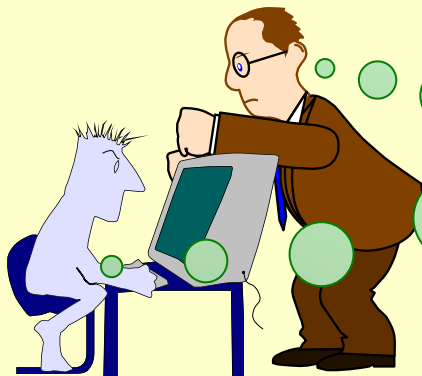
③ `S->Next = TmpCell`

👉 **Top:** `return S->Next->Element`

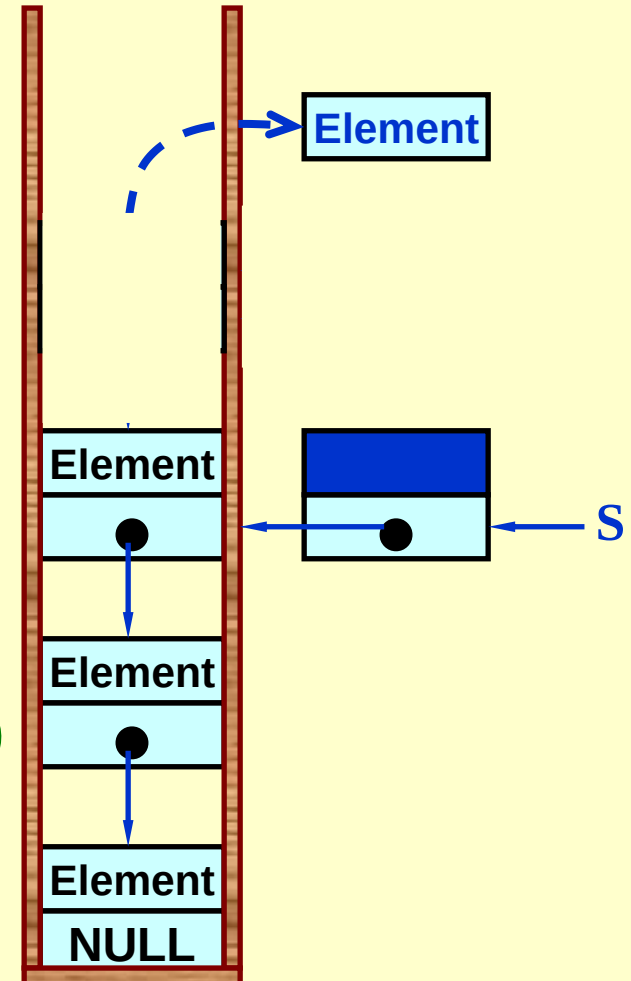
👉 **Pop:** ① `FirstCell = S->Next`

② `S->Next = S->Next->Next`

③ `free ( FirstCell )`



Easy! Simply keep
another stack as
a **recycle bin**.



带回收栈的链栈展示

进栈与出栈序列：5，8，出，7，出，出，2，5，9，出，1
需新创建几个结点？

Experiment 3-1 链栈实现

函数:

```
Stack InitStack(Stack S,int n);  
Status StackTraverse(Stack S);  
Status ClearStack(Stack S);  
ElemType Top(Stack S);  
Status IsEmpty(Stack S);  
Status Push(Stack S, ElemType e);  
ElemType Pop(Stack S);
```

Experiment 3-2 帶回收棧的鏈棧

函数:

Stack InitStack(Stack S,int n);

Status StackTraverse(Stack S);

Status ClearStack(Stack S1,Stack S2);

ElemType Top(Stack S);

Status IsEmpty(Stack S);

Status Push(Stack S1, Stack S2, ElemType e);

Status Recycle(Stack S1, Stack S2);

ElemType Pop(Stack S1,Stack S2);

➤ 顺序栈 Array Implementation (Sequence Stack)

定义：栈的顺序存储结构简称为**顺序栈**，利用一组地址连续的存储单元一次来存放自栈底到栈顶数据元素，同时设置指针top指示栈顶元素在顺序栈的位置。

```
#define STACK_INIT_SIZE 100 //初始化栈的大小
#define STACKINCREMENT 10 //increment 增量
```

```
typedef struct{
    SElemtype *base; /* the base pointer 栈底 */
    SElemtype *top; /* the top pointer 栈顶 */
    int stacksize; /* size of stack */
}
```

```
SqStack ,*PSqStack;
```

```
PSqStack S;
```

```
SqStack s;
```

```
s.base s->base S.base S->base ?
```

```
*s.top *S->top ?
```

```
s.top==s.base S->top==S->base?
```

指针与数组

```
int a[10];
```

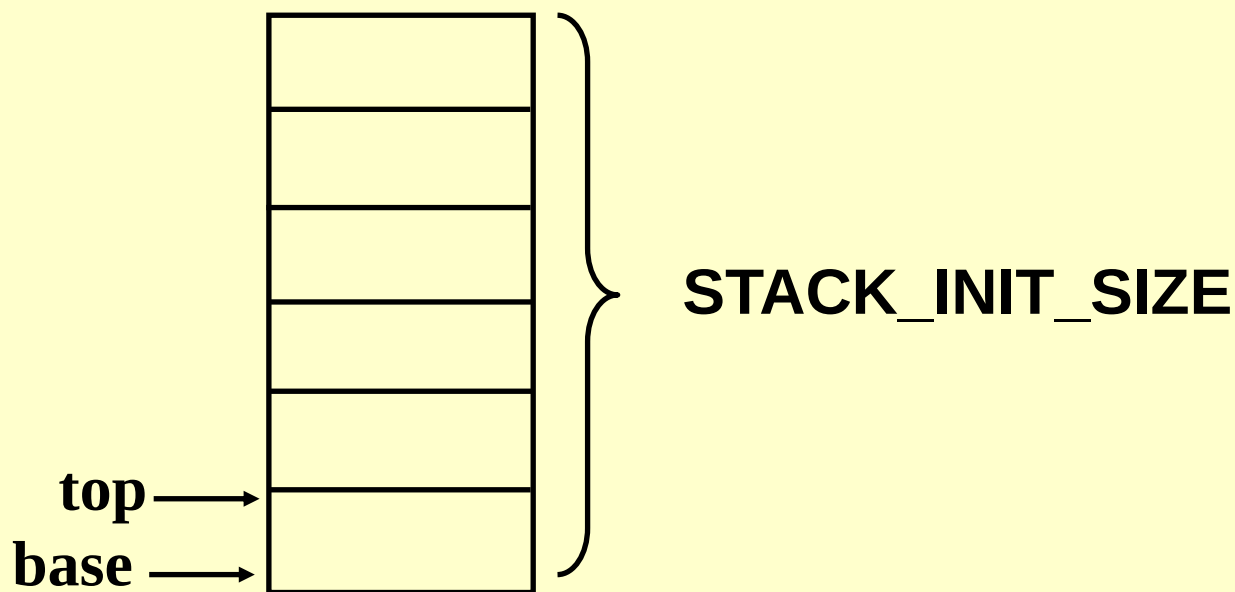
```
int *p;
```

```
p= (int*)malloc(10*sizeof(int));
```

为什么用这种方式？

(1) 初始化:

`S->base = (SElemType *)malloc(STACK_INIT_SIZE*sizeof(SElemType));`



栈顶指针 `S->top == S-> base`, 栈空, 此时出栈为下溢
(underflow)

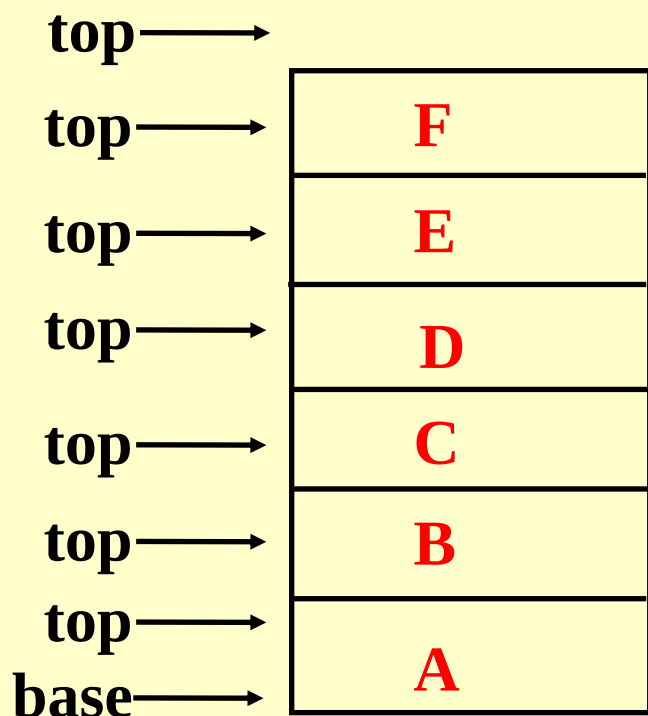
栈底 指针 `S-> base` 为 NULL 表示栈不存在;


```
Status InitStack(PSqStack S)  
{  
    S->base = (SElemType  
    *)malloc(STACK_INIT_SIZE*sizeof(SElemType));  
    if (!S) exit(OVERFLOW);  
    S->top = S->base ;  
    S->stacksize = STACK_INIT_SIZE;  
    return OK;  
}
```

(2) 入栈:将入栈的数据存入top所指空间, top上移.

实现: * S-> top++=e。

若S-> top==S->base+stacksize, 则栈满, 此时入栈,
则上溢 (overflow)



full

*S->top=e;
S->top++;

p=p+1

p++

++p

*p=*p+1 (*p)++

*p++ *(p++)

*++p

Status Push(PSqStack S, SElemType e)

{

if (S->top - S->base >= **S->stacksize-1){**

S->base = _____; //再分配空间

if (!S->base) exit(OVERFLOW); //存储分配失败

S->top = _____; //栈顶位置赋值

S->stacksize = _____; //栈大小

}

_____; //入栈

return OK;

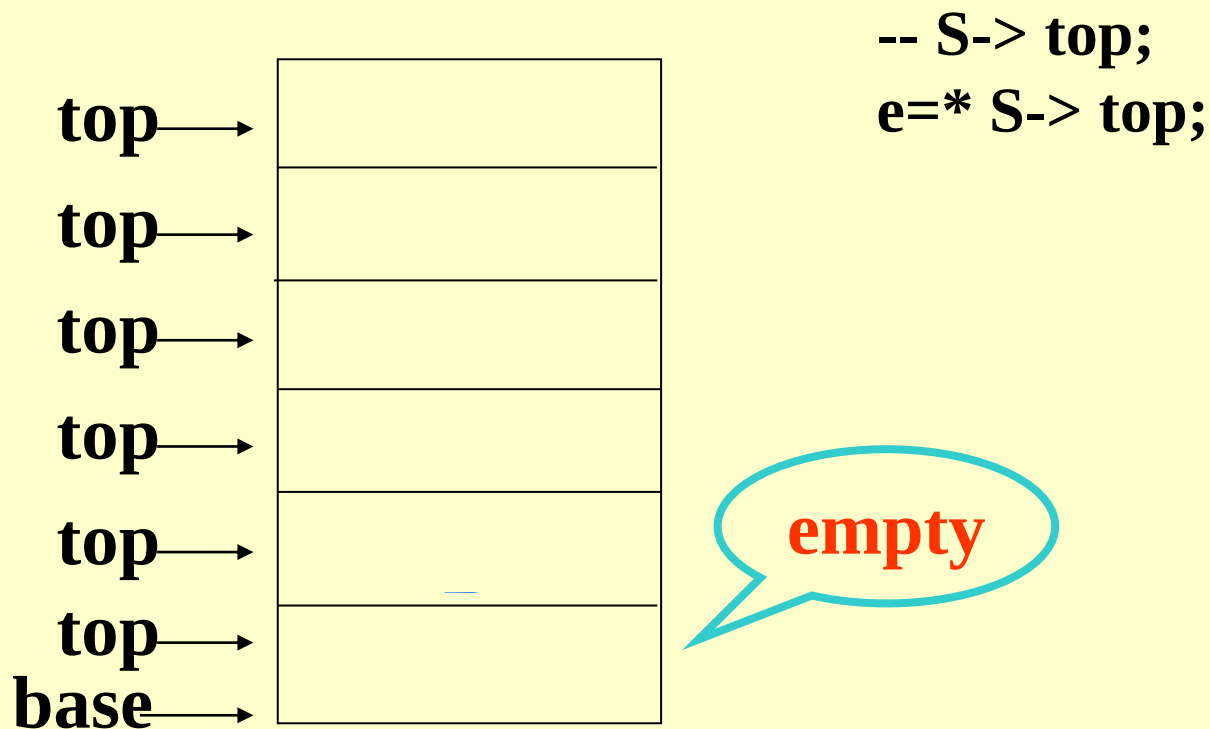
}

S->base = (SElemType *) realloc(,);

(3) 出栈: **top**指针下移, 取出栈顶值。

实现: **e=*-- S-> top;**

当**S-> top== S-> base**时, 栈空。



SElemType Pop(PSqStack S)

{

if (_____) //空栈时出栈

return ERROR;

return _____;//出栈

}

(4) 判空栈

Status IsEmpty(PSqStack S)

```
{  
    if( _____ )  
        return TRUE;  
    else return FALSE;  
}
```

(5) 判断栈满

int StackFull (SqStack S){

```
    return ( _____ );  
}
```

(6) 取栈顶元素

SElemType GetTop(PSqStack S){

```
    if( _____ )  
        return ERROR ;  
    return _____ ;  
}
```

Experiment 3-3 顺序栈实现

函数:

Status InitStack(PSqStack S);

Status StackTraverse(PSqStack S);

Status ClearStack(PSqStack S);

SElemType GetTop(PSqStack S);

IsEmpty(PSqStack S);

Status Push(PSqStack S, SElemType e);

SElemType Pop(PSqStack S);

§ 3.2 应用

✳ 3.2.2 符号（括号）平衡

检查(), [], { } 是否平衡 (力扣20. 有效的括号)

Algorithm { //提前返回 FALSE

Make an empty stack S;

while (read in a character c) {

if (c is an opening symbol)

 Push(c, S);

else if (c is a closing symbol) {

if (S is empty) { **return** FALSE; }

else { /* stack is okay */

if (Top(S) doesn't match c) {**return** FALSE; }

else Pop(S);

 } /* end else-stack is okay */

 } /* end else-if-closing symbol */

} /* end while-loop */

if (S is empty) **return** TRUE;

else return FALSE;

}

$T(N) = O(N)$
where N is the length
of the expression.
This is an
on-line algorithm.

3.18 检查(), [], { } 是否平衡

```
int BracketCorrespondency(char *a)
{
    int i = 0;
    SqStack s;
    SElemType x;
    InitStack(&s);
    while(a[i]){
        switch(a[i]){
            case '(': Push(&s, a[i]); break;
            case '[':      ; break;
            case ')': x=GetTop(&s);
                if (x == '(') Pop(&s);
                else return FALSE;
                break;
            case ']':      ;
                ;
                ;
            default: break;      }
        i++;    }
    if (s.top == s.base) return TRUE;
    return FALSE;
}
```

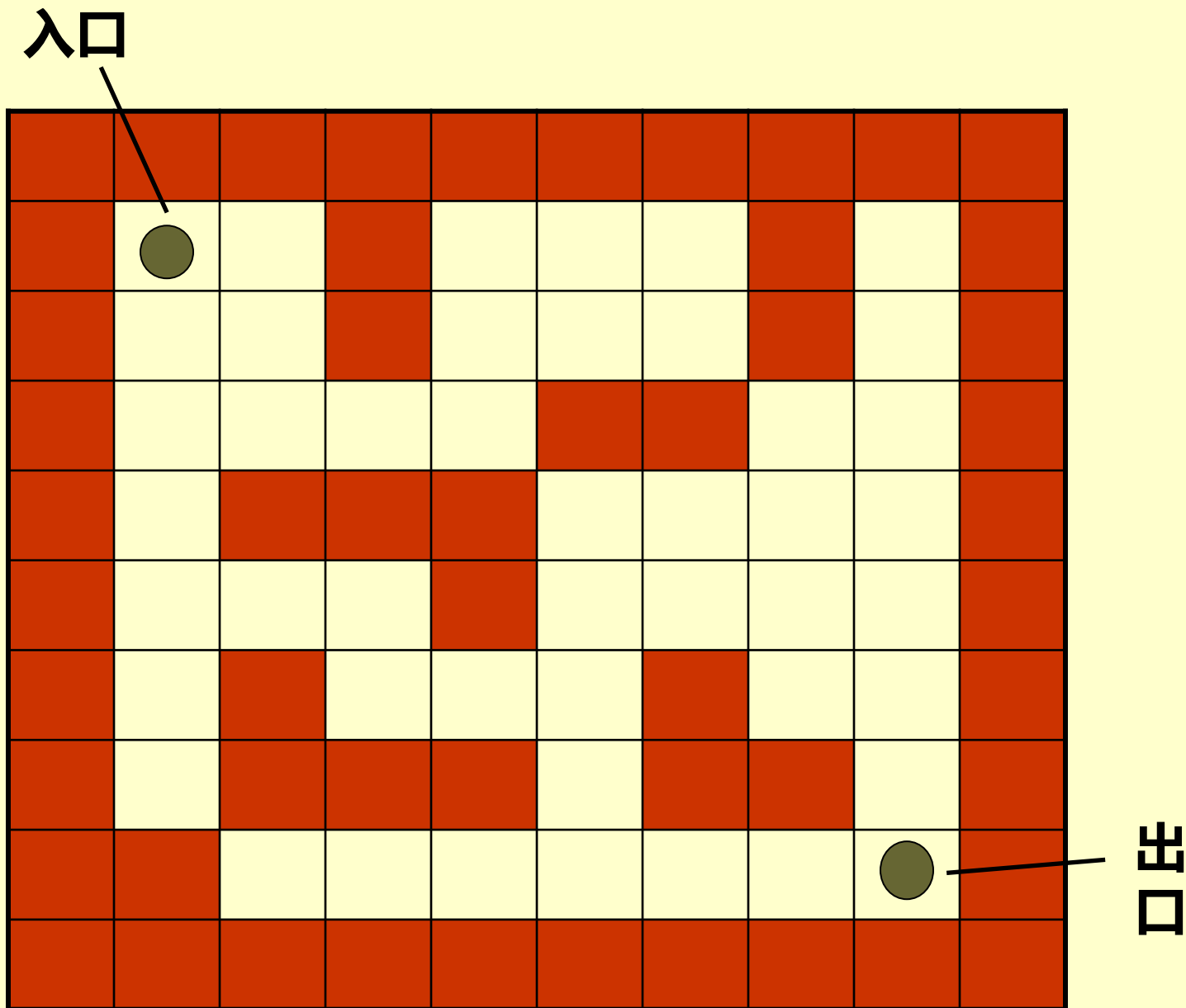
Experiment 3-4 括号匹配检查

问题:

1. 将整数型栈改写为字符型栈;
2. 写函数检查 `()`, `[]`, `{ }` 是否匹配;

3.2 应用

❄ 3.2.4 Maze Solving 迷宫求解 力扣490. 迷宫



问题： 在迷宫中寻找通路以到达出口。

思想： 回溯法, 不断试探且及时纠正错误的搜索方法

步骤：

- 从入口出发, 按某一方向向前探索, 若能走通 (未走过的), 即某处可达, 则到达新点, 否则试探下一方向;
- 若所有的方向均没有通路, 则沿原路返回前一点, 换下一个方向再继续试探; 直到所有可能的通路都探索到或找到一条通路, 或无路可走又返回到入口点。

返到哪个点, 试探哪个方向?

- ◆ 按次序保存所有走过的点, 以及这些点试探过方向;
- ◆ 需要用一个栈保存所能够到达的每一点的下标及从该点前进的方向。

Experiment 3-5 a 迷宫求解递归算法

```
int visit(int i, int j)
{
    //该点走过，标记为1
    maze[i][j] = '1';
    //走到终点，成功
    if(_____)
        success = 1;
    //向四个方向递归调用函数visit()
    if (success != 1 && maze[i][j + 1] == ' ') visit(i, j + 1);
    if (success != 1 && maze[i + 1][j] == ' ') visit(i + 1, j);
    if(_____) visit(____);
    if(_____) visit(____);
    //该点走过，但没成功，则该点重新置为空
    if(success != 1)
        _____;
    return success;
}
```

```
int visit(int, int);
//迷宫矩阵，'#'代表墙壁，' '代表通道
char maze[10][10] = {
    '#', ' ', '#', '#', '#', '#', '#', '#', '#', '#',
    '#', ' ', ' ', '#', ' ', ' ', ' ', '#', ' ', '#',
    '#', ' ', ' ', '#', ' ', ' ', ' ', '#', ' ', '#',
    '#', ' ', ' ', ' ', ' ', '#', '#', ' ', ' ', '#',
    '#', ' ', '#', '#', '#', ' ', ' ', ' ', ' ', '#',
    '#', ' ', ' ', ' ', '#', ' ', '#', ' ', ' ', '#', '#',
    '#', ' ', '#', '#', '#', ' ', '#', '#', ' ', '#',
    '#', ' ', '#', ' ', ' ', ' ', '#', ' ', ' ', '#',
    '#', '#', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
    '#', '#', '#', '#', '#', '#', '#', '#', '#', '#'};
};
int startI = 1, startJ = 1; // 入口
int endI = 8, endJ = 8; // 出口
int success = 0;
```

递归算法

```
typedef struct{
    int order;//通道块在路径上的"序号"
    PosType location;//通道块在迷宫中的"坐标位置"
    int direction;//从此通道块走向下一通道块的"方向"
}SElemType; //栈元素类型
```

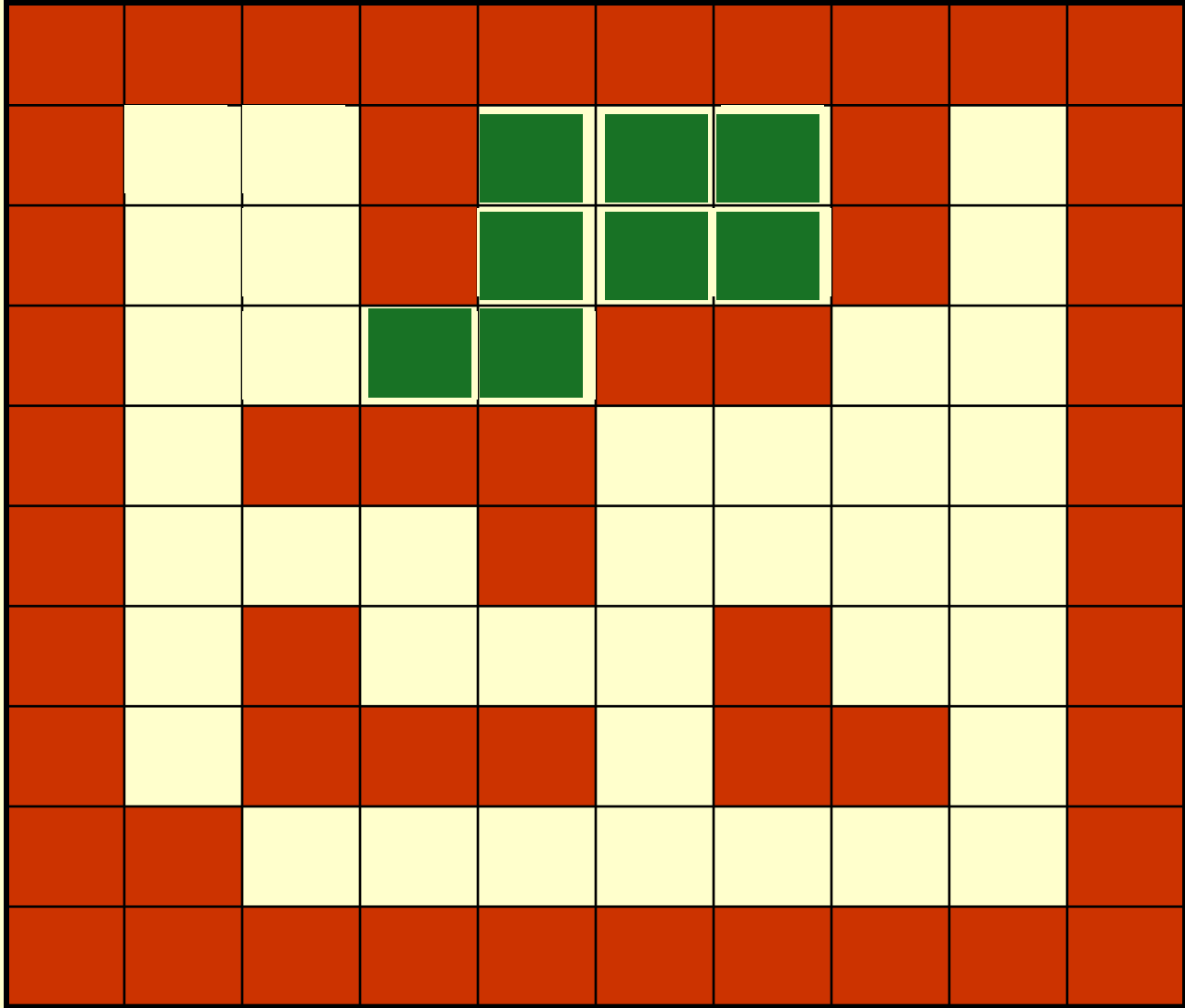
非递归算法

4

3

2

1



4	2	3	3
3	2	2	2
2	2	1	2
1	1	1	

```
typedef struct PosType{  
    //迷宫位置坐标  
    int x;  
    int y;  
}PosType;
```

```
typedef struct SElemType{  
    int order;//通道块在路径上的"序号"  
    PosType location;//通道块的"坐标位置"  
    int direction;//从此通道向下一通道块的"方向"  
}SElemType; //栈元素类型
```

```
typedef struct SqStack{//顺序栈结构定义  
    SElemType *base;  
    SElemType *top;  
    int stacksize;  
}SqStack;
```

通路为栈中的元素组成的路径

➤ 数据结构

```
typedef struct MazeType{//迷宫类型  
    char **maze;//迷宫数据，二维字符数组  
    int **footprint;//足迹数据，二维整型数组  
    int row;//行数  
    int column;//列数  
}MazeType;
```

MazeType *M

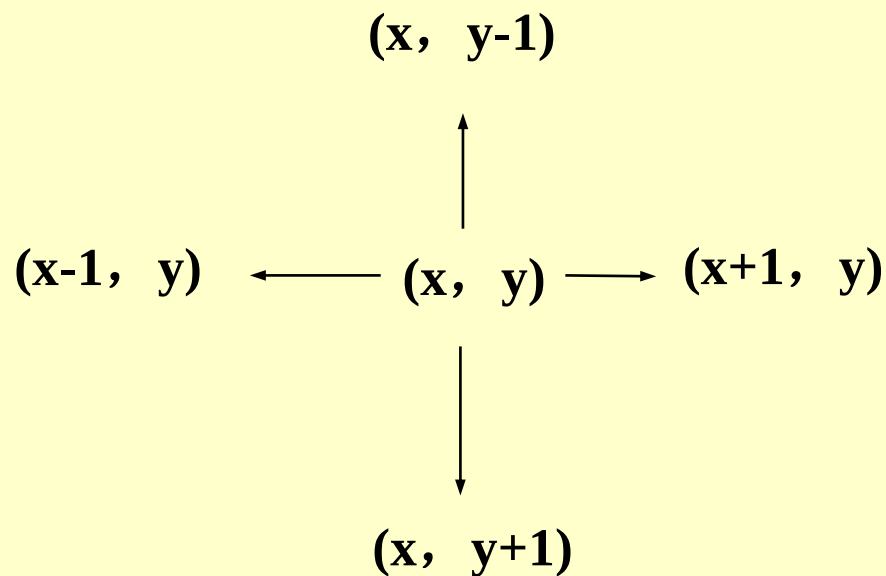
//M->footprint[pos.x][pos.y]=1 走过 0未走过

```

PosType NextPos(PosType pos,int d){
    //获取pos位置d方向的位置
    switch(d){
        case 1://东
            pos.x++;
            break;
        case 2://南
            pos.y++;
            break;
        case 3://西
            pos.x--;
            break;
        case 4://北
            pos.y--;
            break;
        default:
            printf("位置编号出错.\n");
    }
    return pos;
}

```

➤ 试探方向



	x	y
1	1	0
2	0	1
3	-1	0
4	0	-1


```
Status MazePath(SqStack *S,MazeType maze,PosType start,PosType end){
```

```
    int curstep=1;//当前步数
```

```
    SElemType e;
```

```
    PosType curpos=start;//当前位置
```

```
    InitStack(S);//初始化栈
```

```
    do{
```

```
        if(当前位置可以通过){
```

```
            将迷宫的当前位置pos设置为“走过”; // FootPrint
```

```
            e=NewSElem(curstep,curpos,1);//e.direction=1
```

```
            Push(S,e);          //入栈一个元素
```

```
            if(到达终点(出口){ //当前位置为终点位置
```

```
                输出迷宫路径; return TRUE;
```

```
            }
```

```
            走下一步; 步数加一;
```

```
        }//if
```

```
    else{//当前位置不能通过
```

```
        if(!StackEmpty(*S)){
```

```
            Pop(S,&e); //退出上一步的记录
```

```
            while (四个方向都走过&&栈非空){ //上一步的记录中四个方向是否还有可走的，都走过再出栈
```

```
                FootPrint(&maze, e.location); Pop(S,&e); //将迷宫M的pos位置，设为已走
```

```
            }//while
```

```
            if (e.direction<4){ //退出上一步的记录中四个方向还有可走的，换下一个方向
```

```
                e.direction++;
```

```
                Push(S,e);
```

```
                curpos = _____; //当前位置设为下一个位置
```

```
            }//if
```

```
        }//if
```

```
    }//else
```

```
    }while(!StackEmpty(*S));
```

```
    return FALSE;
```

```
}
```

/*若迷宫maze中存在从入口start到出

口end的通道，则求得一条存放在栈中

（从栈底到栈顶），并返回TRUE;

否则返回FALSE */

Experiment 3-5 b 迷宫求解非递归算法

```
SElemType NewSElem(int step,PosType pos,int d){  
    //创建新结点，用step， pos， d初始化该结点  
    SElemType e;  
    e.order=step;  
    e.location = pos;  
    e.direction = d;  
    return e;  
}
```

```
Status FootPrint(MazeType *M,PosType pos){  
    //将迷宫的当前位置pos设置为"走过"， 即footprint该位置为1  
    if((pos.x>M->row)||(pos.y>M->column))  
        return FALSE;  
    M->footprint[pos.x][pos.y]=1;  
    return TRUE;  
}
```

Status Pass(MazeType *M,PosType pos){//判断当前位置是否可通，即为走过的通道块

if((M->row<pos.x)||(M->column<pos.y)){

printf("位置越位.\n");

exit(0);

}

if((0==M->footprint[pos.x][pos.y])&&(M->maze[pos.x][pos.y]==' ')) //可以通过且没走过

return TRUE;

else

return FALSE;

}

* 3.2.4 表达式求值 力扣150. 逆波兰表达式求值

【Example】 An **infix**中缀 expression: $a + b * c - d / e$

A **prefix**前缀 expression: $- + a * b c / d e$

A **postfix**后缀 expression: $a b c * + d e / -$

Reverse Polish notation

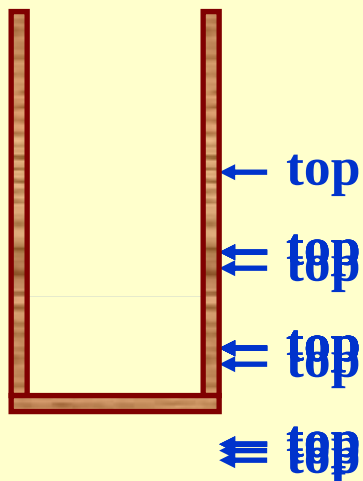
逆波兰式

Operand
操作数

operator with
the highest
precedence优先级

Operator
操作符

【Example】 $6\ 2\ /\ 3\ -\ 4\ 2\ *\ +\ =\ 8$



Get token: 6 (operand)	Get token: 2 (operand)
Get token: / (operator)	Get token: 3 (operand)
Get token: - (operator)	Get token: 4 (operand)
Get token: 2 (operand)	Get token: * (operator)
Get token: + (operator)	Pop: 8

$T(N) = O(N)$.

No need to know precedence rules.

不需要知道优先级规则

* Infix to Postfix Conversion中缀到后缀的转换

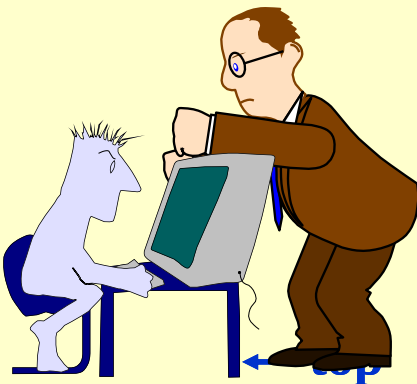
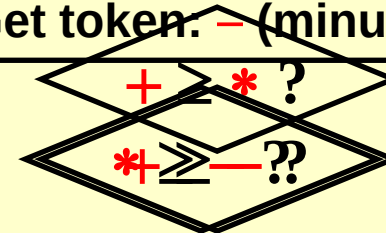
【Example】 $a + b * c - d = a b c * + d -$

Note:

- 判断栈顶操作符与新来一个操作符的优先级；
- $\geq$ 判断为真时，表示栈顶不低于新的操作符，则 出栈；判断为假时进栈。

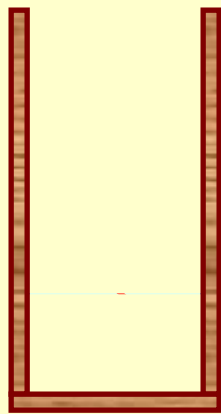
Output: $a b c * + d -$

Get token: a (operand)	Get token: $+$ (plus)
Get token: b (operand)	Get token: $*$ (times)
Get token: c (operand)	Get token: $-$ (minus)
Get token: d (operand)	



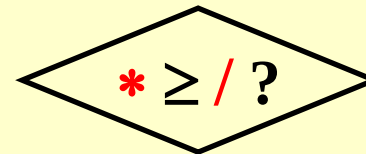
[[Example]] $a * (b + c) / d = a b c + * d /$

Output: $a b c + * d /$



← top

Get token: a (operand)	Get token: $*$ (times)
Get token: $($ (lparen)	Get token: b (operand)
Get token: $+$ (plus)	Get token: c (operand)
Get token: $*$ (times)	Get token: $/$ (divide)
Get token: d (operand)	



NO!! $T(N) = O(N)$

Solutions:

- 只有左括号(进栈，右括号)不进栈；
- 当处理右括号)时才将左括号(出栈；
- 当(不在栈中，当遇见一定进栈即有最高的优先级；但(在栈中时有最低的优先级，当遇见右括号)时一定将(出栈。

由小于10的数字组成的后缀算式 $9\ 3\ /\ 5\ +$ 的值为_____，
中缀算式 $X - Y / 4 + 5$ 对应的后缀算式为_____。

中缀表达式 $(X + Y) * 6 - 5$ 对应的后缀表达式为_____。
由小于10的数字组成的后缀表达式 $7\ 8\ 2\ /\ -\ 3\ +$ 的值为_____。

Experiment 3-6 表达式转换 力扣150+241.

Problems:

```
typedef struct opt
{ char op;
  int level;
}opt;

typedef struct op_stack //定义操作符栈
{ opt st[MAXLEN];
  int top;
}op_stack;

typedef struct D_stack //定义值栈
{ double D[MAXLEN];
  int top;
}D_stack;
```

//-----中缀变后缀的函数-----

```
void InfixToPostfix(char *str, char
*expr, char *kkk)
```

//-----后缀求值函数-----

```
double GetPostfixValue(char *le)
```

```
opt A={'+',1};
```

```
opt R={'-',1};
```

```
opt M={'*',2};
```

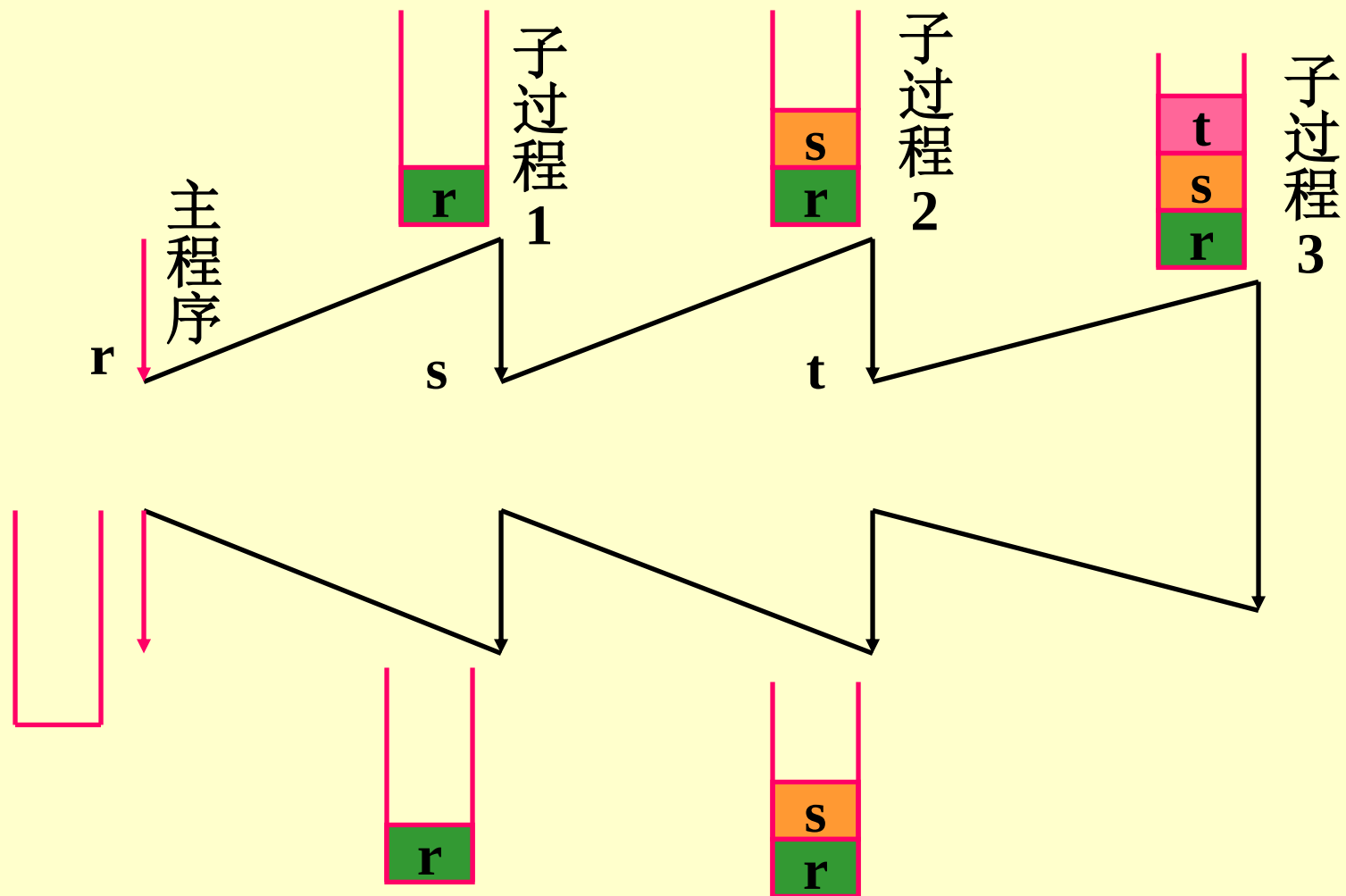
```
opt D={'/',2};
```

```
opt B={'(', -1};
```

```
opt Mo={'%',2};
```

1#2#*3#4#*#+

3.3 栈与递归实现



一、递归及其实现

1、定义：函数直接或间接的调用自身称**递归**

2、递归表达式(递归体)，递归条件(递归出口)

3、实现——建立递归工作栈：

- 类似于多层函数的嵌套调用，调用单位和被调用单位是同一个函数。
- 每次调用系统将属于各个递归层次的信息组成一个**活动记录**：**本层调用的实参、返回地址、局部变量**等信息；
- 活动记录保存在系统的“递归工作栈”中，每当递归调用一次，就要在栈顶为过程建立一个新的活动记录；
- 本次调用结束，则将栈顶活动记录出栈，根据获得的返回地址信息。

4、递归中一定有迭代，迭代中不一定有递归。所有的迭代可以转换为递归，但递归不一定可以转换成迭代。

5、递归效率低，存在栈溢出问题，代码简单。

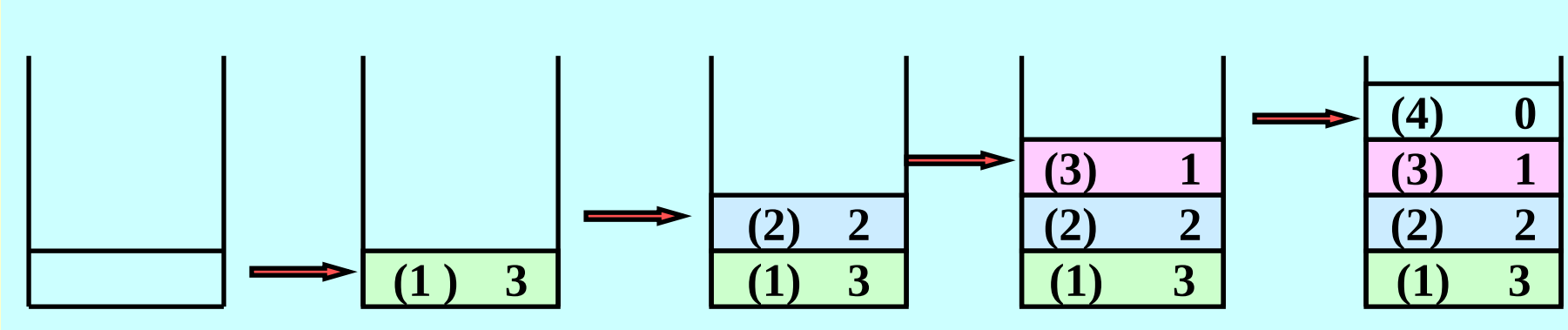
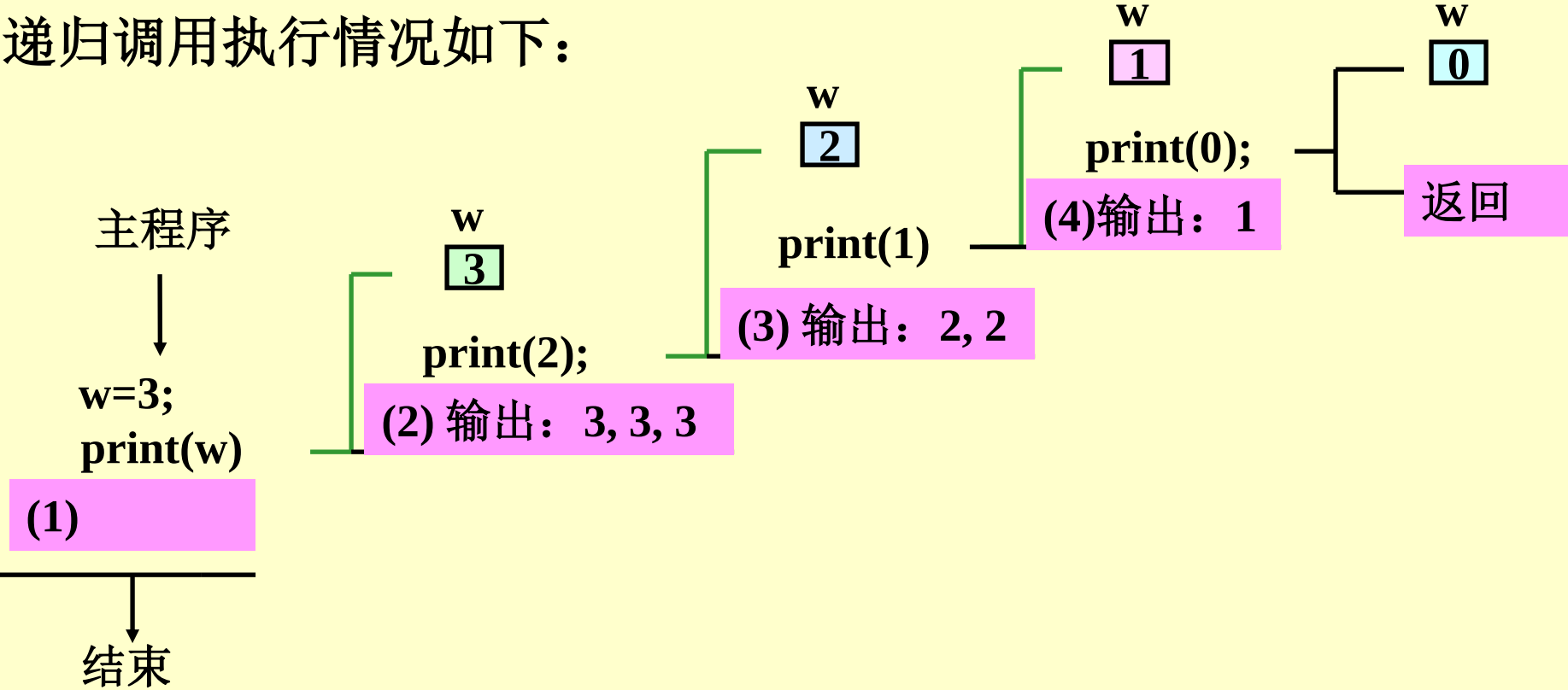
递归的执行情况分析

```
void print(int w){  
    int i;  
    if ( w!=0){  
        print(w-1);  
        for(i=1;i<=w;++i)  
            printf("%3d,",w);  
        printf("\n");  
    }  
}
```

print(3)运行结果:

1,
2, 2,
3, 3, 3,
演示一下

递归调用执行情况如下：



二、Hanoi塔问题 力扣面试题 08.06. 汉诺塔问题

1、传说：

一个印度的古老传说：在世界中心贝拿勒斯（在印度北部）的圣庙里，一块黄铜板上插着三根宝石针。印度教的主神梵天在创造世界的时候，在其中一根针上从下到上地穿好了由大到小的64片金片，这就是汉诺塔。

不论白天黑夜，总有一个僧侣在按照下面的法则移动这些金片：一次只移动一片，不管在哪根针上，小片必须在大片上面。僧侣们预言，当所有的金片都从梵天穿好的那根针上移到另外一根针上时，世界就将在一声霹雳中消灭，而梵塔、庙宇和众生也都将同归于尽。



二、Hanoi塔问题 力扣面试题 08.06. 汉诺塔问题

1、问题描述:

有A, B, C三个塔座，A上套有 n 个直径不同的圆盘，按直径从小到大叠放，形如宝塔，编号1, 2, 3..... n 。要求将 n 个圆盘从A移到C，B可以作为过渡，叠放顺序不变，移动过程中遵循下列原则：

- 每次只能移一个圆盘；
- 圆盘可在三个塔座上任意移动；
- 任何时刻，每个塔座上不能将大盘压到小盘上。

2、解决方法:

- $n=1$ 时, 直接把圆盘从A移到C;
- $n>1$ 时, 先把上面 $n-1$ 个圆盘从A移到B, 然后将 n 号盘从A移到C, 再将 $n-1$ 个盘从B移到C。

将求解 n 个圆盘的Hanoi问题转化为求解 $n-1$ 个圆盘的Hanoi问题, 依次类推, 直至转化成只有一个圆盘的Hanoi问题。

如果考虑一下把64片金片, 由一根针上移到另一根针上, 并且始终保持上小下大的顺序。假设有 n 片, 移动次数是 $f(n)$. 显然: $f(1)=1, f(2)=3, f(3)=7$, 且 $f(n)=2*f(n-1)+1$ 。 $f(n) = ?$ 。

如 n 为64, 一秒钟一次移动, 需多少时间?

舍罕王打算奖赏国际象棋的发明人——宰相西萨·班·达依尔, 麦子

Experiment 3-7 Hanoi

3、算法:

```
main() {  
    int m; printf("Input the number of disks:");  
    scanf("%d",&m);  
    printf("The steps to moving %3d disks:\n",m);  
    hanoi(m,'A','B','C');
```

```
(0) }
```

n	x	y	z	返回地址
---	---	---	---	------

```
(1) void hanoi(int n,char x,char y,char z) {
```

```
(2)     if(n==1)
```

```
(3)         move( x,1,z);
```

```
(4)     else{
```

```
(5)         hanoi(n-1,x,z,y);
```

```
(6)         move( x,n,z);
```

```
(7)         hanoi(n-1,y,x,z);
```

```
(8)     }
```

```
(9) }
```

```
void move(char x, int i, char y)
```

```
{    static int j = 1; //局部变量i声明为 static
```

```
    printf("%d: %d from %c —> %c \n", j++, i, x, y);
```

```
}
```

执行情况:

递归工作栈保存内容:
形参n, x, y, z和返回地址, 返回地址用行编号表示。

演示1

```
void hanoi(int n,char x,char y,char z)
{
    if(n==1)
        move(x,1,z);
    else{
        hanoi(n-1,x,z,y);
        move(x,n,z);
        hanoi(n-1,y,x,z);
    } }
```

```

main()
{
    int m;

    printf("Input number of disks");
    scanf("%d",&m);

    printf("Steps : %3d disks",m);

    hanoi(m,'A','B','C');

```

(0) }

```

void hanoi(int n,char x,char y,char z)

```

```

(1) {

```

```

(2)     if(n==1)

```

```

(3)         move(x,1,z);

```

```

(4)     else{

```

```

(5)         hanoi(n-1,x,z,y);

```

```

(6)         move(x,n,z);

```

```

(7)         hanoi(n-1,y,x,z);

```

```

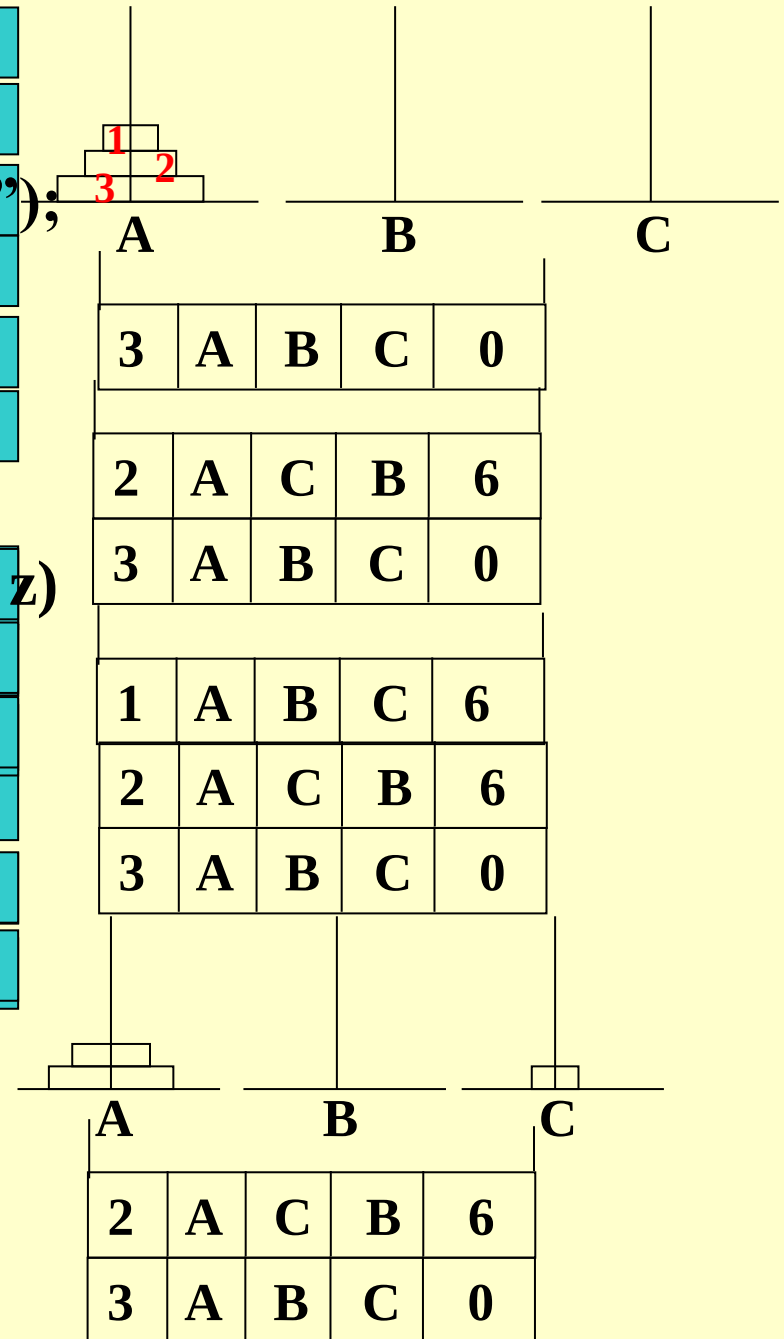
(8)     }

```

```

(9) }

```



main()

{ int m;

printf("Input the number of disks

scanf("%d",&m);

printf("The steps to moving %3d

hanoi(m,'A','B','C');

(0) }

void hanoi(int n,char x,char y,char z)

(1) {

(2) if(n==1)

(3) move(x,1,z);

(4) else{

(5) hanoi(n-1,x,z,y);

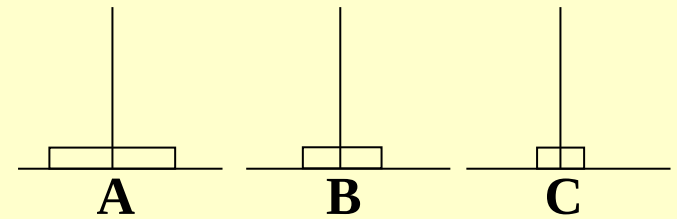
(6) move(x,n,z);

(7) hanoi(n-1,y,x,z);

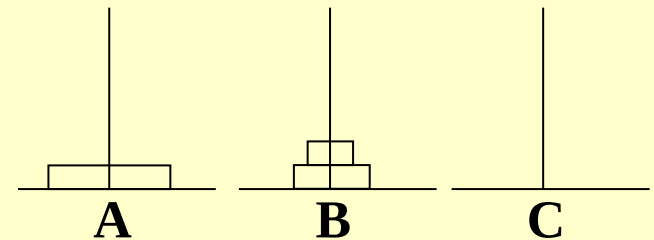
(8) }

(9) }

2	A	C	B	6
3	A	B	C	0



1	C	A	B	8
2	A	C	B	6
3	A	B	C	0



2	A	C	B	6
3	A	B	C	0

3	A	B	C	0
---	---	---	---	---

main()

```
{ int m;
```

```
printf("Input the number of disks
```

```
scanf("%d",&m);
```

```
printf("The steps to moving %3d
```

```
hanoi(m,'A','B','C');
```

```
(0) }
```

```
void hanoi(int n,char x,char y,char z)
```

```
(1) {
```

```
(2)   if(n==1)
```

```
(3)       move(x,1,z);
```

```
(4)   else{
```

```
(5)       hanoi(n-1,x,z,y);
```

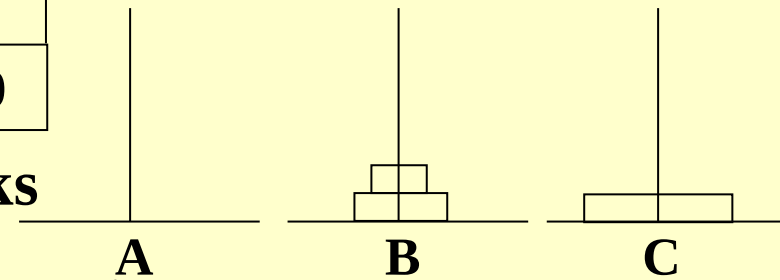
```
(6)       move( x,n,z);
```

```
(7)       hanoi(n-1,y,x,z);
```

```
(8)   }
```

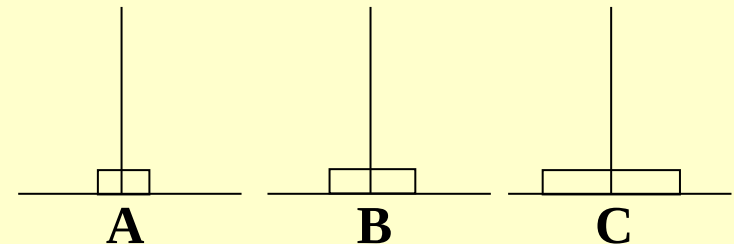
```
(9) }
```

3	A	B	C	0
---	---	---	---	---



2	B	A	C	8
3	A	B	C	0

1	B	C	A	6
2	B	A	C	8
3	A	B	C	0



2	B	A	C	8
3	A	B	C	0

main()

{ int m;

printf("Input the number of disks

scanf("%d",&m);

printf("The steps to moving %3d
hanoi(m,'A','B','C');

(0) }

void hanoi(int n,char x,char y,char z)

(1) {

(2) if(n==1)

(3) move(x,1,z);

(4) else{

(5) hanoi(n-1,x,z,y);

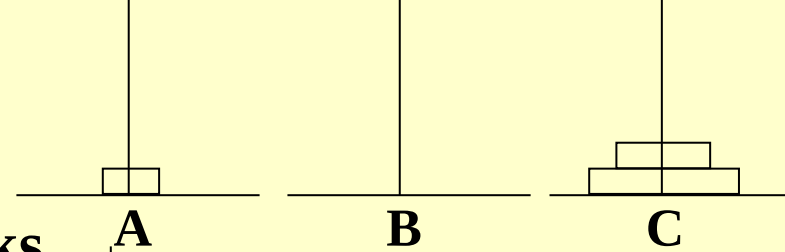
(6) move(x,n,z);

(7) hanoi(n-1,y,x,z);

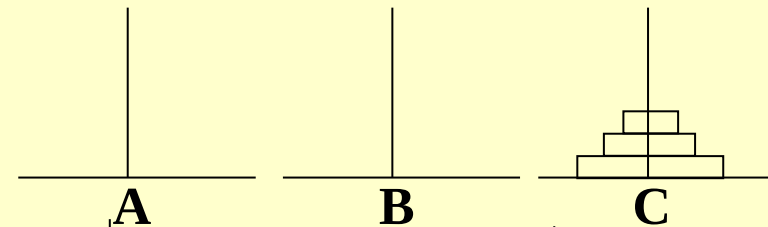
(8) }

(9) }

2	B	A	C	8
3	A	B	C	0



1	A	B	C	8
2	B	A	C	8
3	A	B	C	0



2	B	A	C	8
3	A	B	C	0

3	A	B	C	0
---	---	---	---	---

栈空

返回

美国学者发现一种方法，轮流进行两步操作：设计**非递归算法**

首先把三根柱子按顺序排成品字型，把所有的圆盘按从大到小的顺序放在柱子A上，根据圆盘的数量确定柱子的排放顺序：若 n 为偶数，按顺时针方向依次摆放 A B C；若 n 为奇数，按顺时针方向依次摆放 A C B。

(1)按顺时针方向把圆盘1从现在的柱子移动到下一根柱子，即当 n 为偶数时，若圆盘1在柱子A，则把它移动到B；若圆盘1在柱子B，则把它移动到C；若圆盘1在柱子C，则把它移动到A。

(2)接着，把另外两根柱子上可以移动的圆盘移动到新的柱子上，即把非空柱子上的圆盘移动到空柱子上。当两根柱子都非空时，移动较小的圆盘。这一步没有明确规定移动哪个圆盘，你可能以为会有多种可能性，其实不然，行动是唯一的。

(3)反复进行(1)(2)操作，最后就能按规定完成汉诺塔的移动。

演示2

1、设用链表作为栈的存储结构则进栈操作（ ）。

- (A) 不必判别栈是否为满 (B) 不必判别栈是否为空
(C) 必须判别栈是否为满 (D) 必须判别栈是否为空

2、删除单链表中的由指针p指向的结点，其前一个结点由指针q指向，操作序列为（ ）。

- (A) $q \rightarrow next = p \rightarrow next$; $free(q)$; (B) $p \rightarrow next = q \rightarrow next$; $free(q)$;
(C) $p \rightarrow next = q \rightarrow next$; $free(p)$; (D) $q \rightarrow next = p \rightarrow next$; $free(p)$;

3、设栈S的初始状态为空，现有5个元素组成的序列{1, 2, 3, 4, 5}，对该序列在S栈上依次进行如下操作(从序列中的1开始，出栈后不再进栈)：进栈、进栈、进栈、出栈、进栈、出栈、进栈。试问栈中的元素包括()

- (A) {5, 4, 3, 2, 1} (B) {2, 1} (C) {5, 2, 1} (D) {5, 2, 3}

4、有六个元素6, 5, 4, 3, 2, 1的顺序进栈，下列哪一个不是合法的出栈序列（ ）

- A. 5 4 3 6 1 2 B. 4 5 3 1 2 6 C. 3 4 6 5 2 1 D. 2 3 4 1 5 6

5、假设以数组A[m]存放循环队列的元素,其头尾指针分别为front和rear，则当前队列中的元素个数为（ ）。

- A. $(rear - front + m) \% m$ B. $rear - front + 1$ C. $(front - rear + m) \% m$ D. $(rear - front) \% m$

6、若用一个大小为6的数组来实现循环队列，且当前rear和front的值分别为0和3，当从队列中删除一个元素，再加入两个元素后，rear和front的值分别为多少？（ ）

- A. 1和5 B. 2和4 C. 4和2 D. 5和1

7、设顺序循环队列Q[0: M-1]的头指针和尾指针分别为F和R，头指针F总是指向队头元素的前一位置，尾指针R总是指向队尾元素的当前位置，则该循环队列中的元素个数为（ ）。

(A) R-F (B) F-R (C) (R-F+M)%M (D) (F-R+M)%M

8.设指针变量front表示链式队列的队头指针，指针变量rear表示链式队列的队尾指针，指针变量s指向将要入队列的结点X，则入队列的操作序列为（ ）。

(A) front->next=s; front=s; (B) s->next=rear; rear=s;

(C) rear->next=s; rear=s; (D) s->next=front; front=s;

9. 设带有头结点的单向循环链表的头指针变量为head，则其判空条件是（ ）。

(A) head==NULL (B) head->next== NULL

(C) head->next==head (D) head!= NULL

10. 用S表示进栈操作，用X表示出栈操作，若元素的进栈顺序是1234，为了得到1342的出栈顺序，相应的S和X的操作序列为（ ）。

A、 AXSXSSXX B、 SSSXXSXX

C、 SXSSXXSX D、 SXSSXSXX

11. 【2009统考真题】设栈S和队列Q的初始状态均为空，元素abcdefg依次进入栈S。若每个元素出栈后立即进入队列Q，且7个元素出队的顺序是bdcfeag，则栈S的容量至少是（ ）。

A、 1 B、 2 C、 3 D、 4

§ 3.4 队列 ADT

1. queueADT

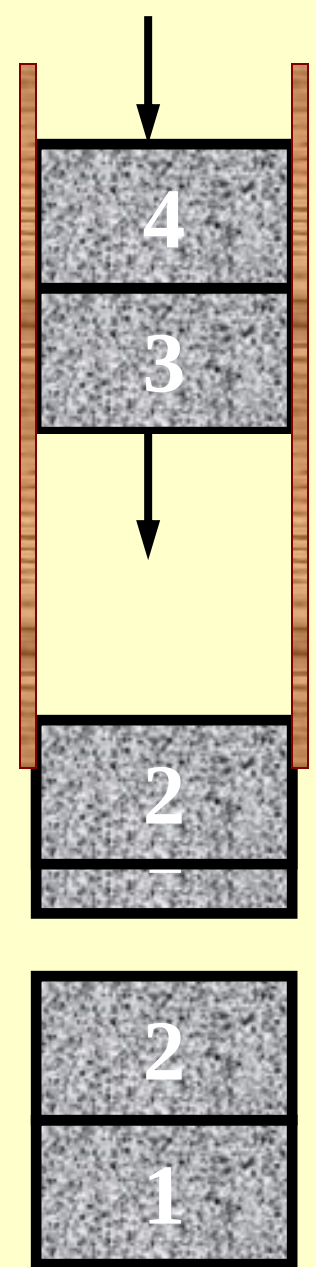
队列是限定只能在表的一端进行插入，在表的另一端进行删除的线性表。**队尾(rear)**——允许插入的一端 **队头(front)**——允许删除的一端 队列特点：先进先出(**FIFO**)（队列亦称作先进先出(First In First Out)的线性表，简称FIFO表）

Objects:元素.

Operations:

- ☞ `int IsEmpty( Queue Q );`
- ☞ `Queue CreateQueue();`
- ☞ `DisposeQueue( Queue Q );`
- ☞ `MakeEmpty( Queue Q );`
- ☞ `★Enqueue( ElementType X, Queue Q );` 入队
- ☞ `ElementType★Front( Queue Q );`
- ☞ `★Dequeue( Queue Q );` 出队

【Example】 Job Scheduling in an Operating System



2.队列的顺序表示和实现

```
#define MAXQSIZE 100
```

```
typedef struct{
```

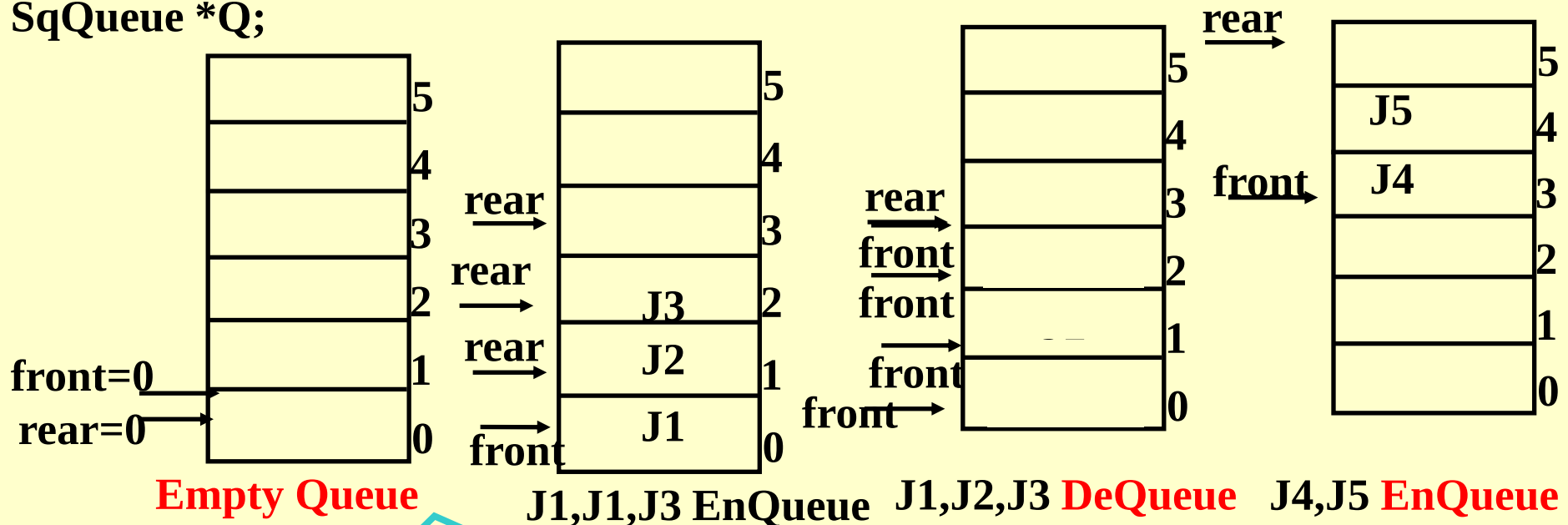
```
    QElemtype *base;
```

```
    int front; /* 队头指针the front pointer , 也可定义为指针型*/
```

```
    int rear; /* 队尾指针the rear pointer */
```

```
}SqQueue;
```

```
SqQueue *Q;
```



Two pointers: front, rear
 rear points to the rear next
 front points to the front
 初始化: $Q \rightarrow \text{front} = Q \rightarrow \text{rear} = 0$

空队: $Q \rightarrow \text{front} == Q \rightarrow \text{rear}$
 EnQueue: $Q \rightarrow \text{base}[Q \rightarrow \text{rear}++] = x;$
 DeQueue: $x = Q \rightarrow \text{base}[Q \rightarrow \text{front}++];$

Circular Queue 循环队列:

给定数组大小为M:

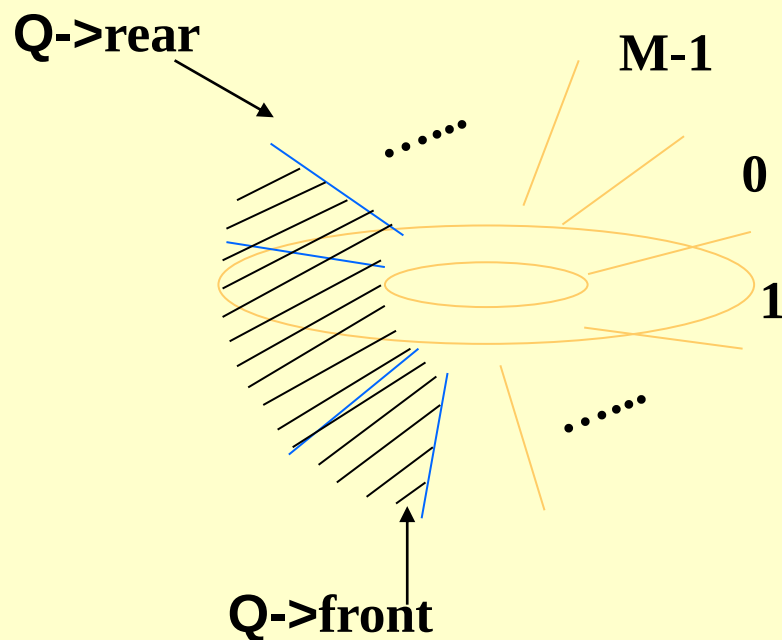
- 当 $\text{front} == 0, \text{rear} == M-1$ 时, 再有元素入队发生溢出——**真溢出**
- 当 $\text{front} \neq 0, \text{rear} == M-1$ 时, 再有元素入队发生溢出——**假溢出**

思想: Suppose the array is a circle, $\text{base}[0]$ follows $\text{base}[M-1]$, and if $\text{rear}+1 == M$, then $\text{rear} = 0$;
将数组看成一个圆环, $\text{base}[0]$ 接到 $\text{base}[M-1]$ 后。

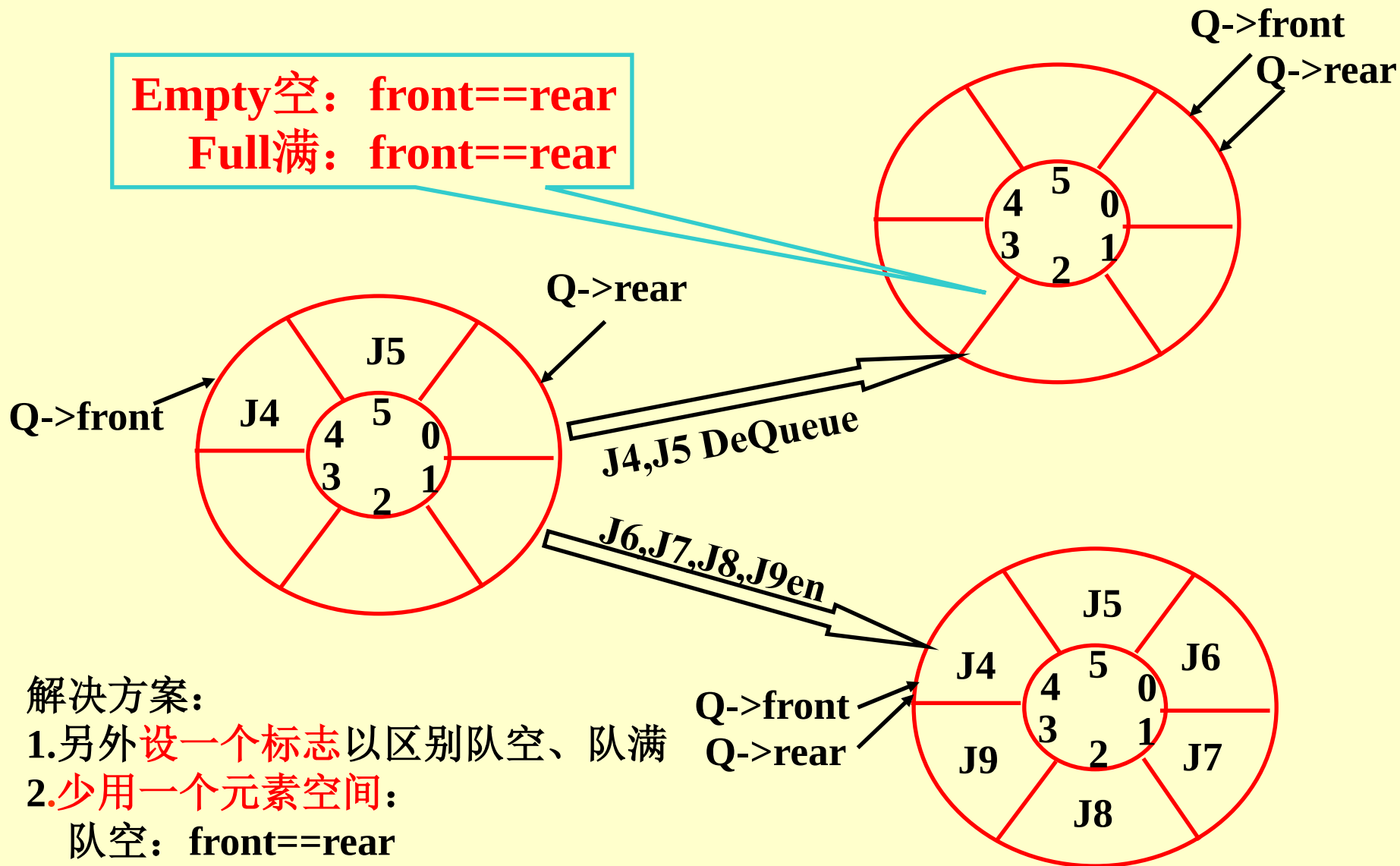
方法: modular arithmetic 模运算

Enqueue: $Q \rightarrow \text{base}[Q \rightarrow \text{rear}] = x$;
 $Q \rightarrow \text{rear} = (Q \rightarrow \text{rear} + 1) \% M$;

Dequeue: $x = Q \rightarrow \text{base}[Q \rightarrow \text{front}]$;
 $Q \rightarrow \text{front} = (Q \rightarrow \text{front} + 1) \% M$;



Empty空: $front == rear$
Full满: $front == rear$



解决方案:

1. 另外**设一个标志**以区别队空、队满
2. **少用一个元素空间:**

队空: $front == rear$

队满: $(rear + 1) \% M == front$

3. 是使用一个计数器记录队列中元素的总数
(实际上是队列长度)

Operations:

(1) 置空队

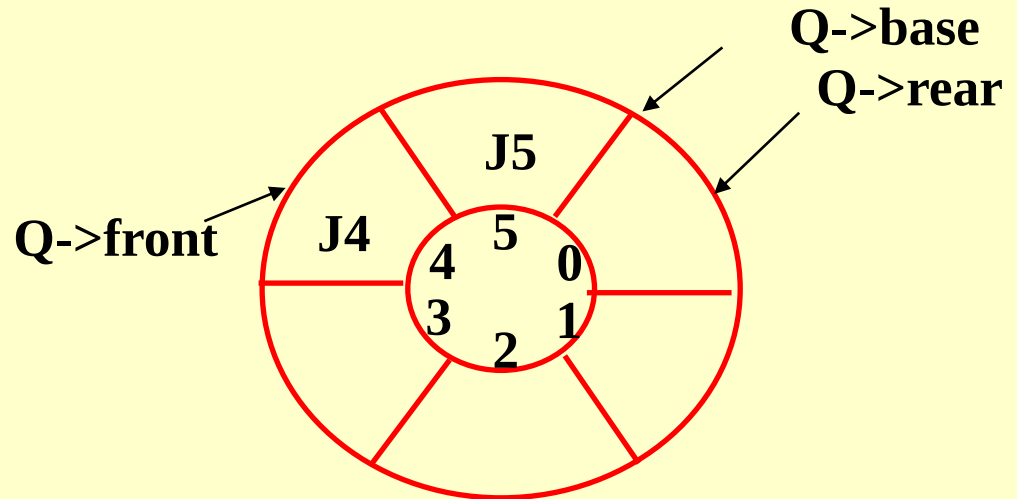
```
Status InitQueue(SqQueue *Q){  
    Q->base=(QElemType *) malloc(MAXQSIZE*sizeof(QElemType))  
    if (!Q->base) exit (OVERFLOW);  
    Q->front=Q->rear=0;  
    return OK;  
}
```

(2) 返回队列长度

```
int QueueLength(SqQueue *Q) {  
    return (Q->rear-Q->front +MAXQSIZE)%MAXQSIZE;  
}
```


(3) 入队 EnQueue

```
Status EnQueue(SqQueue *Q, QElemType e){  
    if((Q->rear+1)%MAXQSIZE==Q->front )  
        return ERROR;  
    _____;  
    _____;  
    return OK;  
}
```



(4) 出队DeQueue

QElemType DeQueue(SqQueue *Q)

{

QElemType e;

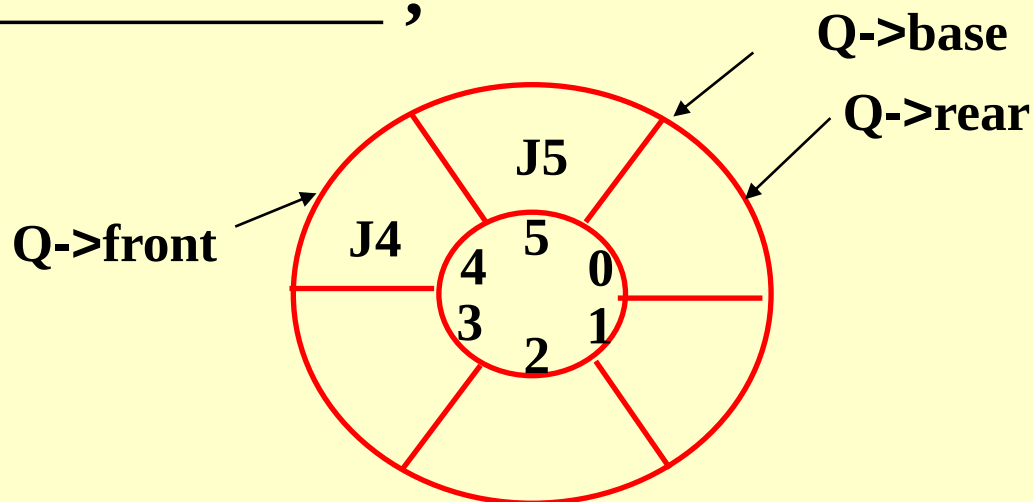
if(Q->front==Q->rear) return ERROR;

_____ ;

_____ ;

return e;

}



(5) 取队头元素 Get the front element

QElemType QueueFront (SqQueue *Q)

{

if(queueEmpty(Q))

error(“queue is empty.”);

return Q->base[Q->front];

}

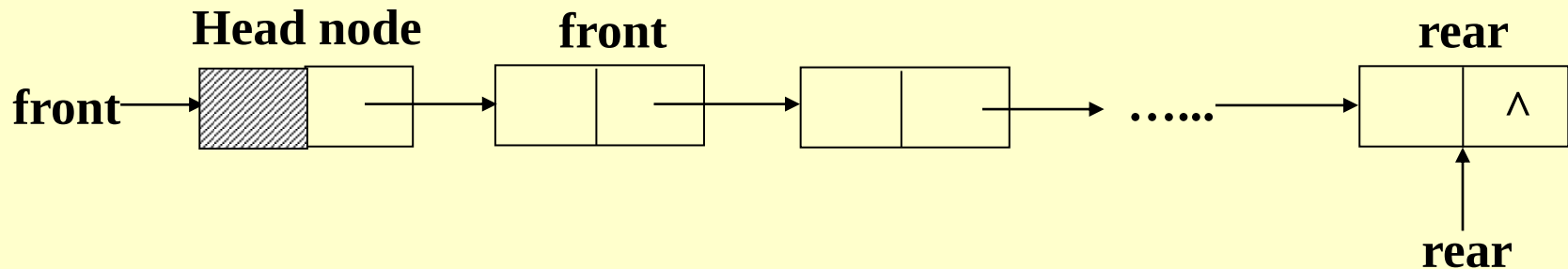
2. 链队Linked list implementation of Queues

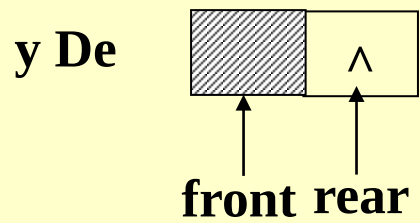
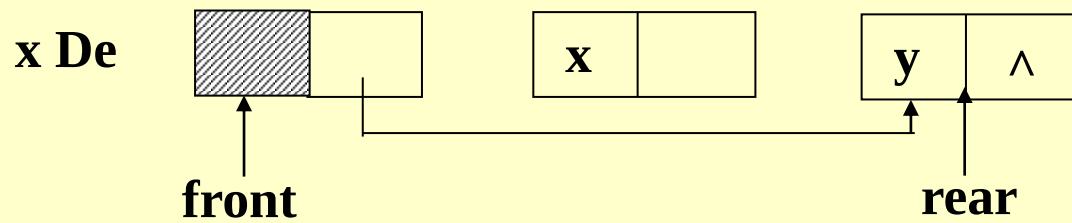
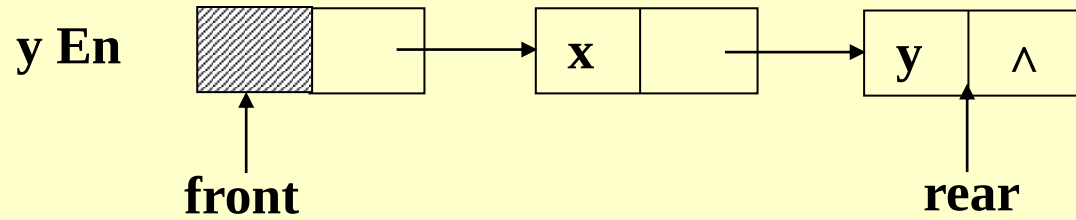
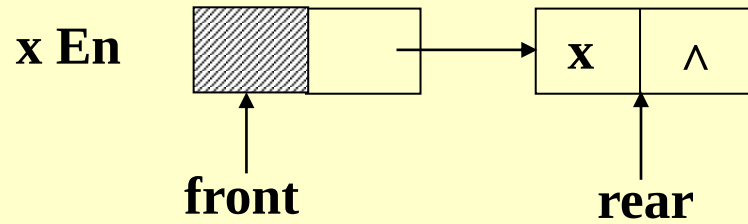
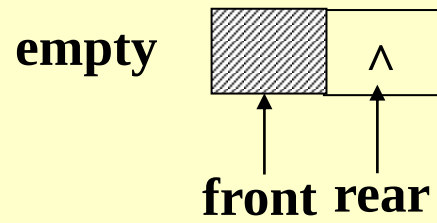
(Linked list implementation is trivial)

结点定义Node

```
typedef struct QNode{  
    QElemtype data;  
    struct QNode *next;  
}QNode, *Queueptr;
```

```
typedef struct {  
    queueptr front, rear;  
} LinkQueue ;
```





2. 双端队列

deque（double-ended queue，双端队列）是一种具有队列和栈的性质的数据结构。双端队列中的元素可以从两端弹出，其限定插入和删除操作在表的两端进行。双向链表或数组实现

力扣641. 设计循环双端队列

- 双端队列是限定插入和删除操作在表的两端进行的线性表。铁道转轨网络来比喻双端队列。
- 可以有输出受限的双端队列（即一个端点允许插入和删除，另一个端点只允许插入的双端队列）和输入受限的双端队列（即一个端点允许插入和删除，另一个端点只允许删除的双端队列）。
- 如果限定双端队列从某个端点插入的元素只能从该端点删除，则蜕变为两个栈底相邻的栈。

力扣239. 滑动窗口最大值

描述：给定整数数组 `nums`，大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。滑动窗口每次只向右移动一位，返回每次滑动窗口中的最大值。

输入：nums = [1,3,-1,-3,5,3,6,7], k = 3

输出：[3,3,5,5,6,7]

解释：

滑动窗口的位置

最大值

[1	3	-1]	-3	5	3	6	7	3
----	---	-----	----	---	---	---	---	---

1	[3	-1	-3]	5	3	6	7	3
---	----	----	-----	---	---	---	---	---

1	3	[-1	-3	5]	3	6	7	5
---	---	-----	----	----	---	---	---	---

1	3	-1	[-3	5	3]	6	7	5
---	---	----	-----	---	----	---	---	---

1	3	-1	-3	[5	3	6]	7	6
---	---	----	----	----	---	----	---	---

1	3	-1	-3	5	[3	6	7]	7
---	---	----	----	---	----	---	----	---

力扣239. 滑动窗口最大值-思路

- 遍历数组，将可能的窗口内最大值存放在有序双向队列（单调减）中，并用 L,R 来标记窗口的左边界和右边界。队列中保存的数组中的数从大到小排序。(或者数组下标)
- 初始时L和R为0，有形成窗口的过程，此过程没有最大值，L不动，R向右移。当窗口大小形成时，L和R一起向右移。
- 每次移动时，判断当前遍历数据与队首的值相等则出队，否则不出队；同时，如果当前遍历的数比队尾的值大，则需要弹出队尾值，直到队列重新满足从大到小。当前窗口的最大值即为队首的数。

nums = [1,3,-1,-3,5,3,6,7], k = 3

[1, 3, -1, -3, 5, 3, 6, 7]

- i=0,nums[0]=1。队列为空,直接加入。{1}
- i=1,nums[1]=3。队尾值1, 3>1, 弹出队尾值, 加入3。队列: {3}
- i=2,nums[2]=-1。队尾值3, -1<3, 直接加入。队列: {3,-1}。窗口形成, L=0, R=2, result=[3]
- i=3,nums[3]=-3。队尾值-1, -3<-1, 直接加入。队列: {3,-1,-3}。队首3对应的下标为1, L=1,R=3, result=[3,3]
- i=4,nums[4]=5。队尾值为-3, 5>-3, 依次弹出后加入。队列: {5}。L=2,R=4, result=[3,3,5]
- i=5,nums[5]=3。队尾值5, 3<5, 直接加入。队列: {5,3}。此时L=3,R=5 result=[3,3,5,5]
- i=6,nums[6]=6。队尾值为3, 6>3, 依次弹出后加入。队列: {6}。L=4,R=6, result=[3,3,5,5,6]
- i=7,nums[7]=7。队尾值为6, 7>6, 弹出队尾值后加入。队列: {7}。L=5,R=7 result=[3,3,5,5,6,7]

链表List

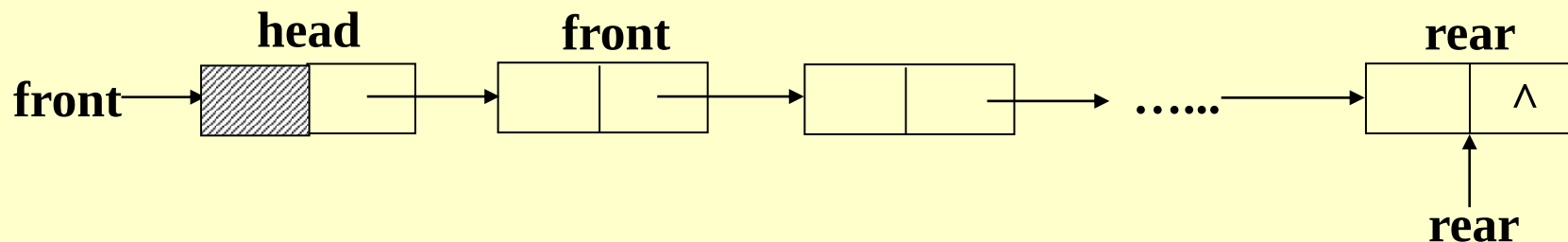
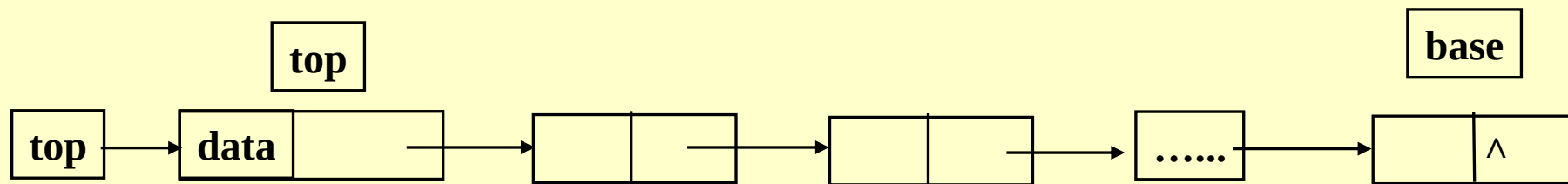
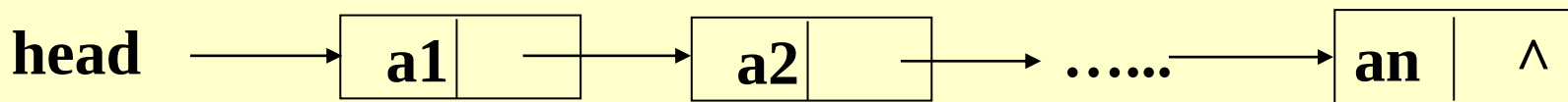
链栈linked list stack

链队linked list queue

```
typedef struct LNode {  
    ElemType data;  
    struct LNode *next;  
} LNode, *LinkList;  
  
typedef struct LStackNode{  
    ElemType data ;  
    struct LStackNode *next;  
} LStackNode
```

```
typedef struct {  
    queueptr front, rear;  
} LinkQueue ;
```

e.g.: LinkList p;



Experiment 3-8 顺序队列

Problems:

Status InitQueue(SqQueue *Q);//Initialize an empty list

Status QueueEmpty(SqQueue *Q);

Status EnQueue(SqQueue *Q, QElemType e);

QElemType DeQueue(SqQueue *Q);

Status QueueTraverse(SqQueue *Q);

Experiment 3-9 链队列

Problems:

Status InitQueue(LinkQueue *Q);

//Initialize an empty list

Status QueueEmpty(LinkQueue *Q);

Status EnQueue(LinkQueue *Q, QElemType e);

QElemType DeQueue(LinkQueue *Q);

Status QueueTraverse(LinkQueue *Q);

Experiment 3-10 顺序循环双端队列

/*双端队列（641. 设计循环双端队列），单调队列（从大到小）（239. 滑动窗口最大值）实现*/

Status InitQueue(cirDeque *Q);

Status insertFront(cirDeque *Q, QElemType e);

Status insertRear(cirDeque *Q, QElemType e);

Status deleteFront(cirDeque *Q);

Status deleteRear(cirDeque *Q);

QElemType getFront(cirDeque *Q);

QElemType getRear(cirDeque *Q);

Status isEmpty(cirDeque *Q);

Status isFull(cirDeque *Q);

Status QueueTraverse(cirDeque *Q);

Status DeMonoQueue(cirDeque *Q, QElemType e);

Status EnMonoQueue(cirDeque *Q, QElemType e);

QElemType maxMonoQueue(cirDeque *Q);

int * maxSlidingWindow(cirDeque *Q, int *nums, int numsSize, int k);